

問題&心得:

- (1) 在這次實驗遇到的問題微算機開發板在LED亮燈一樣，會有兩顆燈依舊無法正常亮起，後念更換實驗版後也是有一顆無法正常亮，但不影響操作亮燈使用。
- (2) 在中斷控制部分的石英頻率，我們在預報以及上次操作部分接用了錯誤數據下去計算以及有部分純數學計算錯誤的粗心，那在原先的話是用成舊板子的24.5MHz，但其實實驗板是22.118MHz，經過修正後也得到了正確結果(新版公式(from老師提供的電子書)為: $12/22.118\text{MHz} = 5.425$)。
- (3) 在這次實驗操作後，對於上次以及這次控制LED燈亮燈的操作方法以及撰寫code更加熟悉了，基本沒有其他太大的問題及錯誤。

本次實驗內容結果:

1. lab03基本:

(1) 作法:

- 用Timer0的中斷方式去控制LED亮燈，不用delay延遲；為每4次溢位作為一次週期，LED會依序向左移動來形成所需的跑馬燈效果。
- 在實驗板使用部分是22.118MHz晶振，在Timer部分則是以Mode1(16位元)做運作，主程式的話僅是等待中斷觸發。

(2) 程式撰寫重點:

- ORG 0000H做為初始位址，ORG 000BH則作為Timer0的中斷向量。
- 再宣告PATTERN為LED的顯示資料、CNT4為中斷次數的計數。初始的PATTERN=0FEH，會輸出到 P0 使最右邊LED亮起。
- 後呼叫Timer_Config 設定TMOD=01H、TCON=10H、IE=82H，開啟Timer與中斷；TH0、TL0則設為 00H，CNT4=4。
- 主程式會以SJMP MAIN_LOOP來形成無限迴圈等待做中斷。

(3) 中斷流程:

- 每次Timer0溢位就進入ISR，執行DJNZ CNT4的倒數；未到0則會重新載TH0、TL0後返回。
- 當CNT4為0時就重設為4，PATTERN經RL左移後會輸出至P0，LED就會顯示向左移動(亮燈)。
- 最後重新載入Timer初值(TH0、TL0)並以RETI返回主程式。

- (4) 結果:LED以固定時間間隔依序由右往左亮起，形成穩定的跑馬燈效果，那在速度部分則與主程式無關，而是由Timer0中斷做為更精準的控制。下方是程式碼(+註解)以及結果照片

```

ORG 0000H ; 程式起始位址在0000H
LJMP START ; 一開始跳到主程式 START

ORG 000BH ; Timer0中斷的向量位置(固定在000BH)
LJMP Timer0_ISR ; 當Timer0發生中斷時跳到這個副程式執行

CNT4 EQU 30H ; 設變數CNT4放在內部RAM的30H位置, 用來計數溢位次數
PATTERN EQU 31H ; 設PATTERN放在31H, 用來記錄目前LED顯示的狀態

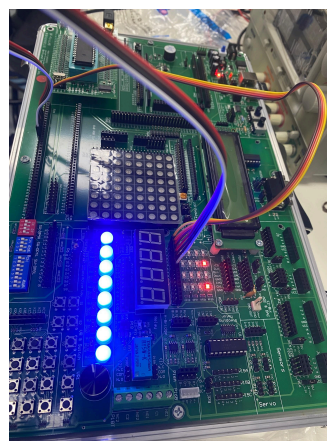
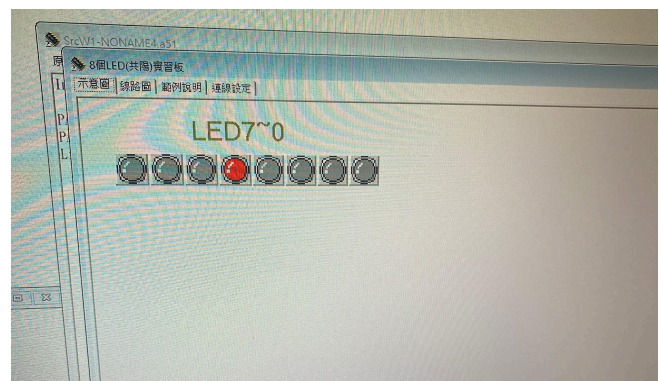
START
MOV PATTERN, #0FEH ; 設初始的LED亮燈狀態(只有最右邊那顆亮)
MOV P0, #0FEH ; 把初始的亮燈狀態輸出到P0埠
ACALL Timer_Config ; 呼叫Timer設定的子程式, 啟動Timer0與中斷功能
MOV CNT4, #4 ; 設定CNT4=4, 表示每四次中斷換一次燈

MAIN_LOOP
SJMP MAIN_LOOP ; 主程式什麼都不做, 一直在這裡等中斷的發生

Timer_Config
MOV TMOD, #01H ; 設定Timer0為模式1(16位元計數)
MOV TCON, #10H ; 啟動Timer0 (TR0=1)
MOV IE, #82H ; 開啟總中斷(EA=1)與Timer0中斷(ET0=1)
MOV TL0, #00H ; 設定Timer低8位初值為00H
MOV TH0, #00H ; 設定Timer高8位初值為00H
RET ; 回到主程式繼續執行

Timer0_ISR
DJNZ CNT4, reset_timer ; 每次中斷CNT4減1, 若還沒減到0就直接跳到reset_timer
MOV CNT4, #4 ; 若CNT4已減到0, 重設回4 (準備下次再數四次)
MOV A, PATTERN ; 把目前的LED狀態放進累加器A
RL A ; 將A裡的LED pattern左移一位, 讓下一顆LED亮
MOV PATTERN, A ; 更新PATTERN成新的亮燈狀態
MOV P0, A ; 把新的亮燈pattern輸出到P0埠(LED會變化)

```



```

reset_timer
MOV    TL0, #00H    ; Timer低8位重新歸零, 準備下一次計時
MOV    TH0, #00H    ; Timer高8位重新歸零
RETI                      ; 結束中斷服務程式, 回到主程式繼續執行

END                      ; 結束

```

2. bonus(以下內容是與基本款不同的部分):

(1) 作法:

- 目標:這次的bonus實驗改以Timer0中斷方式精確控制時間, 讓LED精準以穩定的一秒間隔依序亮起, 產生持續輪流移動跑馬燈的效果。
- a. Timer0設為Mode 1(16 位元), 根據晶振頻率22.118MHz及12T的時序條件, 計算出單次中斷約為0.025秒
- b. 得到設定初始值TH0 = 4CH、TL0 = 1DH。會以每40次中斷(CNT = 28H)即累積約1秒。
- c. 主程式起始時會將WALK = #01H, 最右側的LED亮, 接著進入無限迴圈等待Timer中斷觸發。
- d. 每當Timer0溢位中斷發生, 變數CNT會遞減一次;當 CNT = 0時, 就會重新載入CNT = 40H, 並執行 RL A 使LED向左亮燈移動一格。
- e. 若移動超出範圍則重設回最右端(A = 01H), 輸出前再經CPL A反相以符合共陽極 LED 的亮燈條件。

(2) 結果:整體運作下, LED 能以穩定的節奏每"1"秒輪流亮起, 實現穩定且連續的跑馬燈效果。

```

ORG    0000H    ; 程式從 0000H 開始執行
LJMP   START    ; 跳到主程式 START

ORG    000BH    ; Timer0 中斷向量位址
LJMP   Timer0_ISR ; Timer0 溢位時跳到中斷副程式

TH0_INIT EQU    4CH    ; Timer0 高位初值 (對應約 0.025
秒)
TL0_INIT EQU    1DH    ; Timer0 低位初值
CNT_TARGET EQU    28H ; 每 40 次中斷為 1 秒

CNT      EQU    30H    ; 中斷次數倒數
WALK     EQU    31H    ; 記錄目前哪顆 LED 亮

START:
MOV     WALK, #01H    ; 一開始亮最右邊那顆 LED
MOV     P0, #0FEH    ; 共陽極設定, P0.0 亮
ACALL   Timer_Config  ; 呼叫 Timer 設定副程式
MOV     CNT, #CNT_TARGET ; 設定中斷次數為 40 次

MAIN_LOOP:
SJMP    MAIN_LOOP    ; 主程式不做事, 一直等中斷

Timer_Config:
MOV     TMOD, #01H    ; 設定 Timer0 為 Mode1 (16 位元)
MOV     TCON, #10H    ; 啟動 Timer0 (TR0=1)
MOV     IE, #82H      ; 開啟總中斷與 Timer0 中斷
MOV     TH0, #TH0_INIT ; 載入初值高位
MOV     TL0, #TL0_INIT ; 載入初值低位
RET     ; 回主程式

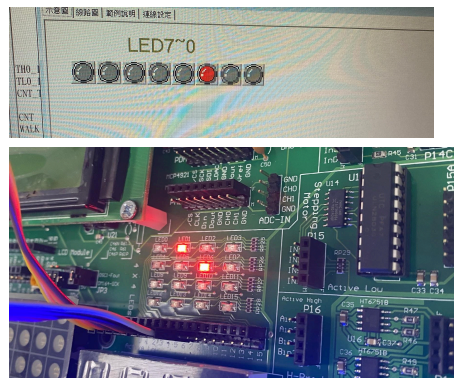
Timer0_ISR:
MOV     TH0, #TH0_INIT ; 每次中斷重新載入初值
MOV     TL0, #TL0_INIT
DJNZ    CNT, ISR_Exit  ; CNT 減 1, 未到 0 就直接返回
MOV     CNT, #CNT_TARGET ; CNT 到 0 時重設為 40 次
MOV     A, WALK        ; 取出目前亮燈資料
RL      A              ; LED 左移一顆
JNZ     NoWrap         ; 若未超出範圍就跳過
MOV     A, #01H        ; 超出範圍回到最右邊

NoWrap:
MOV     WALK, A        ; 更新目前亮燈位置
CPL     A              ; 共陽極需反相輸出
MOV     P0, A          ; 顯示新的 LED 狀態

ISR_Exit:
RETI                      ; 結束中斷, 回主程式

END                      ; 結束

```



根據 C8051F04x 手冊之 Timer 運作原理:

$$\text{Timer 0/1 計時頻率: } f_{\text{timer}} = \frac{\text{SYSCLK}}{12 \times \text{PRESCALE}} = \frac{22.118 \text{ MHz (內聯時脈)}}{12 \times 8051 \text{ 架構下, 每個計時單位 } 12 \text{ machine cycle}}$$

$$\text{每個 tick 時間: } T_{\text{tick}} = \frac{12}{22.118 \text{ MHz}} = 0.5429 \mu\text{s}$$

⇒ Mode1 為 16-bit 計數 ⇒ 0 → 65535 後就溢位
 ⇒ 得單次溢位時間: $65535 \times 0.5429 \mu\text{s} \approx 0.0355 \text{ (sec)}$ (不預載的情況下)

$$\Rightarrow \text{要 1 秒} \approx 40 \text{ 次中斷} \Rightarrow T_{\text{overflow}} = \frac{1 \text{ s}}{40} = 0.025 \text{ s}$$

$$\Rightarrow \therefore \text{Timer 計時 } 0.025 \text{ s 就溢位} \Rightarrow N_{\text{counts}} = \frac{0.025 \text{ s}}{0.0005429 \text{ s}} \approx 46083$$

$$\Rightarrow \text{PRELOAD} = 65535 - 46083 = 19452 = 0x4C1D \Rightarrow \therefore \begin{cases} \text{TH0} = 4CH \\ \text{TL0} = 1DH \end{cases}$$

以上計算為更改正確版(預報的是錯誤版本)